

Project Proposal - Adaptive SSH Honeypot for Real-World Attack Telemetry Collection on AWS

Track Title - [Ghost Cloud: LLM Honeypots in AWS](#)

Intern Name - [Shikhar Sahay](#)

GOALS & PROJECT OBJECTIVES

The primary objective of this project is to design and deploy a securely isolated AWS-hosted SSH honeypot capable of attracting real-world threat actors and collecting meaningful attack telemetry. Rather than focusing solely on building a deceptive SSH environment, the project aims to operationalize the honeypot as a telemetry collection system that captures attacker behavior such as credential attempts, executed commands, session metadata, filesystem interactions, and overall session activity. The implementation will primarily leverage the Beelzebub framework as the foundation for realistic SSH-based attacker interaction.

A secondary objective is to evaluate whether adaptive LLM-assisted SSH responses can improve attacker engagement when compared to traditional static honeypots. The project will additionally explore session persistence mechanisms, allowing returning attackers to encounter previously modified environments in order to improve realism and reduce suspicion. The overall goal is to generate higher-quality attack telemetry and derive practical security insights from observing real-world attacker behavior against cloud-exposed infrastructure.

SYSTEM ARCHITECTURE OVERVIEW

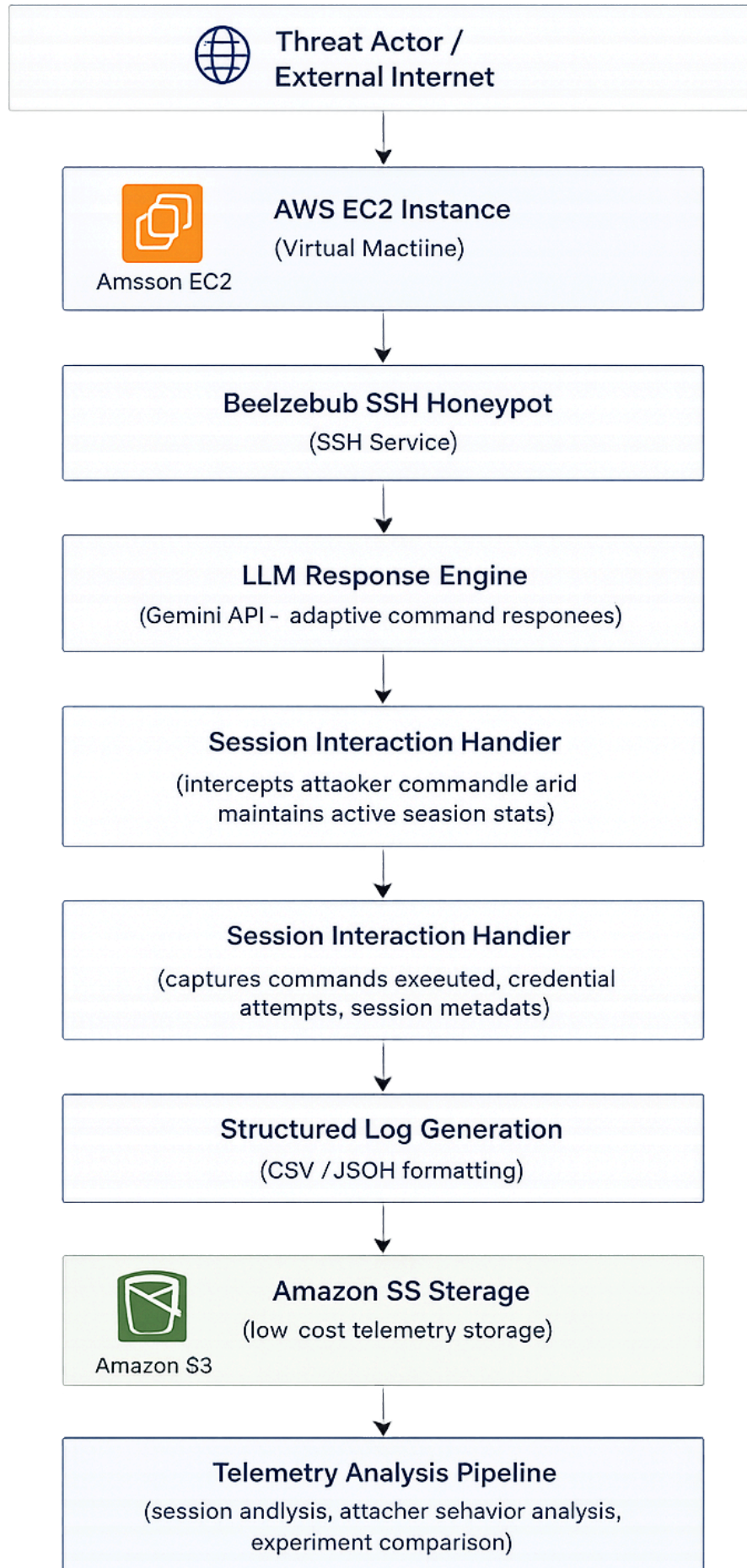
The proposed system architecture, illustrated in the figure below, is designed to safely expose an SSH-accessible cloud environment capable of attracting

real-world attackers while ensuring that all interactions remain securely isolated from the underlying infrastructure. The system will be deployed on an AWS EC2 instance, where the Beelzebub SSH honeypot framework will serve as the primary interaction layer for accepting incoming SSH connections and emulating a realistic attack surface.

Once an attacker establishes a connection, session activity and executed commands will be routed through an LLM-assisted response engine responsible for generating adaptive and context-aware SSH interactions in order to improve realism and prolong attacker engagement. Simultaneously, a session interaction handler and telemetry logging layer will capture critical attack data, including credential attempts, executed commands, session metadata, and filesystem activity.

This collected telemetry will then be structured in JSON or CSV format and stored in Amazon S3 for low-cost persistent storage, allowing the collected attack data to be analyzed later for identifying attacker behavior patterns, measuring interaction depth, and comparing the effectiveness of static versus adaptive honeypot behavior across different deployment experiments.

This architecture prioritizes telemetry quality, operational safety, and experimental extensibility by ensuring all interactions remain isolated within an EC2-hosted environment with centralized structured logging. The system captures SSH attacker behavior such as authentication attempts, command execution, and session activity for later analysis. The LLM-based response layer improves interaction realism without replacing the core honeypot, enabling controlled comparison between static and adaptive deception strategies. This separation allows the system to function as an experimental platform for analyzing real-world SSH attacker behavior under varying interaction models.



EXPERIMENTAL METHODOLOGY

The project follows a staged experimental design that transitions from a baseline honeypot deployment to progressively more adaptive and persistent interaction models. The primary objective is to evaluate how increasing interaction realism impacts attacker engagement and the quality of collected SSH telemetry.

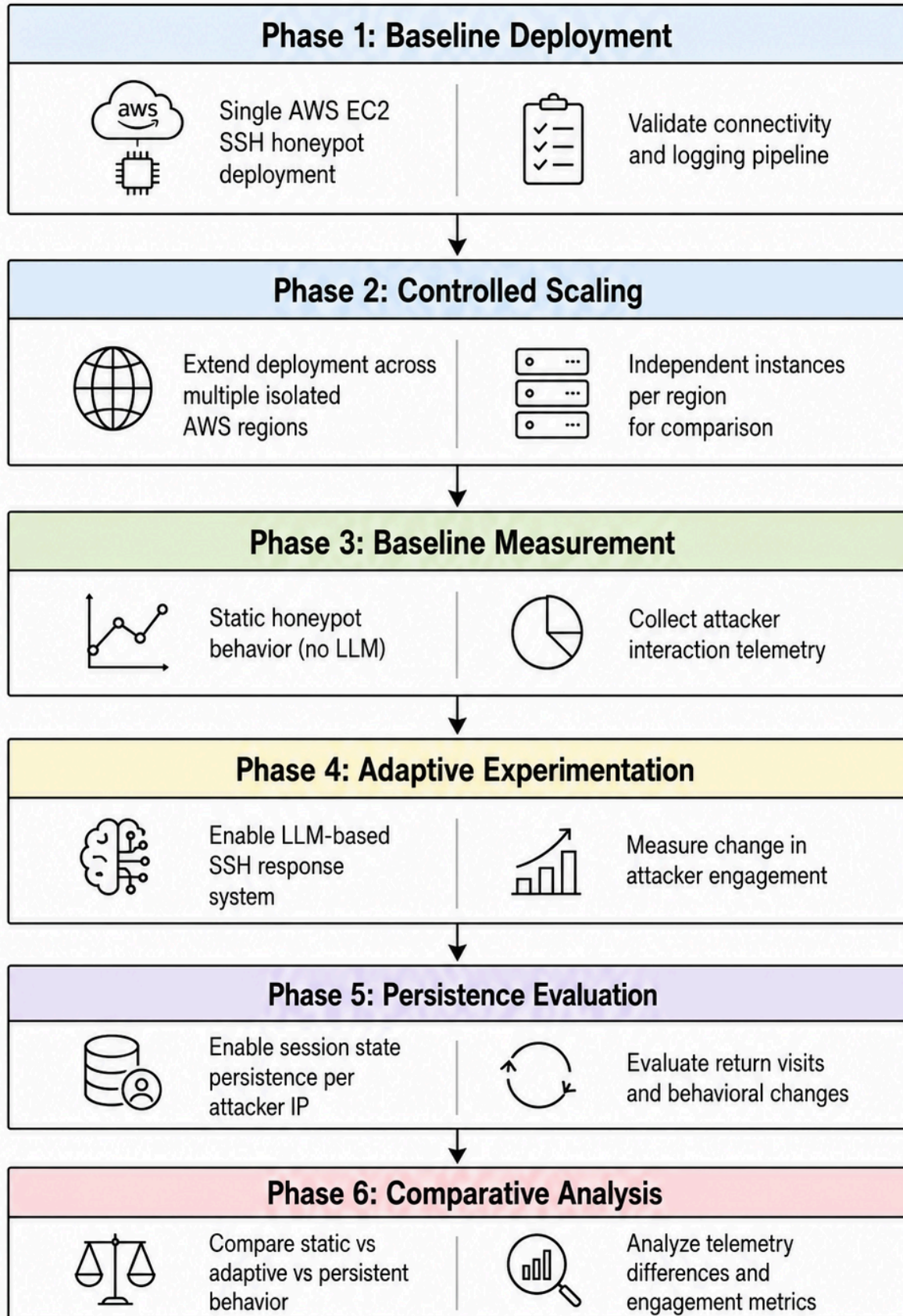
The first phase involves deploying a single SSH honeypot instance on AWS to validate end-to-end functionality, including external accessibility, command capture, credential attempt logging, and structured telemetry storage. Once validated, the deployment will be extended to a small multi-region setup of 3–5 isolated AWS instances, each running independently to capture differences in attacker behavior across geographic regions without shared state interference.

A baseline dataset is first established using a static honeypot configuration without adaptive responses, serving as the control condition against which all subsequent experimental setups are evaluated. Next, an LLM-powered adaptive interaction layer is introduced to generate dynamic SSH responses based on attacker activity, simulating more realistic environments and evaluating whether increased interaction depth leads to higher engagement and longer sessions.

In the final phase, session persistence is enabled to maintain attacker-specific state across reconnects, allowing returning attackers to observe continuity, such as previously created files and system changes. This tests whether a persistent state increases perceived realism and engagement.

Across all phases, structured telemetry is collected for comparative analysis, including session duration, command frequency, credential attempts, interaction depth, and return visits. The goal is to compare static, adaptive, and persistent configurations to assess their impact on real-world SSH attacker behavior.

Experimental Methodology Flow for Adaptive SSH Honeypot Evaluation on AWS



MILESTONES

→ Week 1 (Project Understanding and System Familiarization):

- Gained an understanding of the overall project scope, SSH honeypot behavior, and AWS-based deployment model
- Reviewed Beelzebub architecture, telemetry flow requirements, and existing honeypot frameworks
- Established baseline technical context for system design and experimentation

→ Week 2 (System Design and Deployment Preparation):

- Finalized experimental design and AWS deployment strategy
- Defined logging structure, telemetry format, and region plan
- Prepared baseline configuration for initial honeypot deployment
- Validated overall system flow at a design level

→ Week 3 (Baseline Deployment and Validation):

- Deploy a single SSH honeypot instance on AWS using Beelzebub
- Validate external accessibility and SSH interaction flow
- Capture credential attempts, command execution, and session metadata
- Verify structured telemetry storage in Amazon S3

→ Week 4 (Multi-Region Deployment):

- Extend deployment to 3-5 AWS regions based on baseline stability
- Ensure each instance operates independently without shared state
- Maintain consistent logging and telemetry formats across regions

- Begin observing variation in attacker behavior across regions

→ Week 5 (Static Baseline and Adaptive Integration):

- Run the static honeypot configuration as the control experiment
- Collect baseline attacker behavior metrics for comparison
- Integrate an LLM-powered adaptive response system
- Begin comparative telemetry collection between static and adaptive modes

→ Week 6 (Persistence and Final Analysis):

- Enable session persistence for attacker-specific state across reconnects
- Analyze telemetry across static, adaptive, and persistent configurations
- Compare engagement metrics, including session duration, command frequency, and return visits

RESOURCES/TECHNOLOGY STACK

The following section outlines the core tools and infrastructure used for implementing and evaluating the system. The stack is intentionally kept minimal and cost-aware to ensure efficient deployment, controlled experimentation, and operation within low-cost AWS usage limits.

1. Cloud Infrastructure:

- Amazon Web Services (AWS EC2) for honeypot deployment and isolated environment hosting

- Amazon S3 for structured telemetry storage and long-term data retention

2. Honeypot Framework:

- Beelzebub for SSH honeypot emulation and attacker interaction handling

3. System Components:

- Python for backend logic, telemetry processing, and experiment orchestration
- Redis for session state management and attacker-specific persistence

4. LLM Integration:

- Llama 3.1 for adaptive SSH response generation
- Optional API-based inference layer for comparative experiments

5. Logging and Monitoring:

- AWS CloudWatch for system logs and runtime monitoring

6. Data Format:

- JSON and CSV formats for structured telemetry storage and analysis
-